בס"ד

מצגת סיכום מינ"פ 1 - מינ"פ 2 מבוא להנדסת תוכנה

אפרת ברינקמן & נעמי גרונשטיין

יולי 2025

הסצנה שלנו

הסצנה מציגה חלל מלא בגופים גאומטרים. בשהמרכזי שהבם היא תחנת הקמח עליה השקענו הרבה מאמצים, והמורכבת מכנפיים משולשות תלת מימדיות, מוקפת בגופים שונים תלת מימדיים חלקם מנסים לדמות את 3 מגדלי עזריאלי המוכרים, כל אלא מוצבים על שטח ורקע כהה עם תאורות המשתקפות על פניהם בצורות השונות.

מרכיבי הסצנה:
גאומטריות:

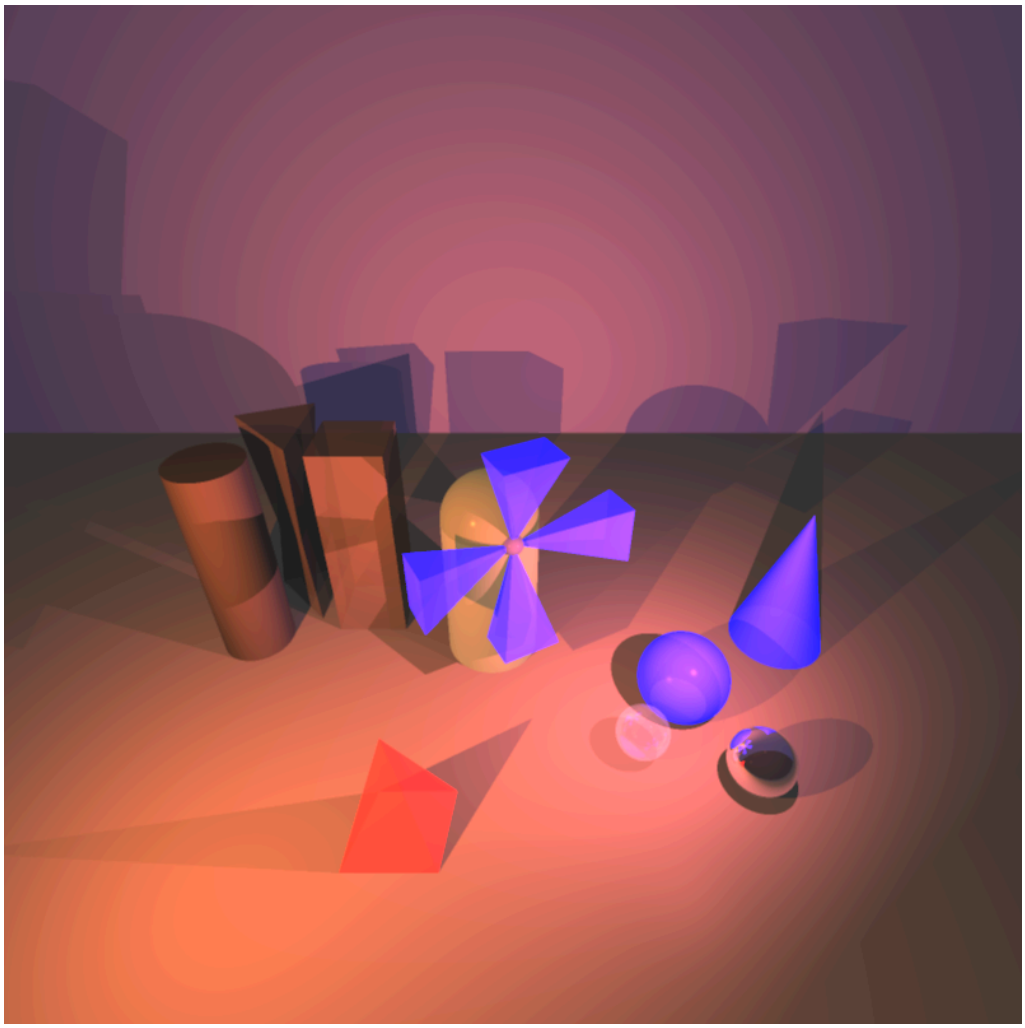
- **בסיס התחנה העשוי מגליל סופי צהוב**
- **גג התחנה עשוי מכדור בעל קוטר, צבע ומקדמי שקיפות זהים לבסיס**
- **טורבינת התחנה מורכבת מ- 4 פרמידות עם בסיס מרובע שקופות חלקית בצבע סגול**
- **כדור קטן לסימון מרכז הטורבינה.**
- **גליל סופי, תיבה מוארכת וב פירמידות בעלות בסיס משולש (דימוי למגדלי עזריאלי)**
- **פירמידה אדומה שקופה חלקית.**
- **חרוט כחול סגול**
- **שלושה בדורים: אחד קריסטלי שקוף, אחד מבריק כהה ומשקף, ואחד בצבעי כנפי התחנה חצי שקוף.**

תאורות:

- **תאורת ספוט מצד ימין: המדמה שמש בשקיעה ,**
- **תאורת ספוט מצד שמאל**

מאפיני רינדור כלליים:

- **רזולוצייה : 800×800 פיקסלים**
- **גודל התמונה 500*500**



מיני פרויקט 1: Anti-Aliasing

הבעיה:

רינדור סטנדרטי עם קרן אחת לכל פיקסל יוצר תופעה של קצוות משוננים, בעיקר בקצוות ובאזורים עם ניגודיות גבוהה. זה יוצר אפקט של מדרגות שפוגע באוטנטיות של התמונה . . .

השיפור: Anti-Aliasing (AA) using Jittered Sampling.

לכל פיקסל נשלחות קרניים רבות למיקומים אקראיים בתוך רשת של פיקסלים (גודל רשת מוגדרת מראש 9*9) , וצבע הפיקסל מחושב כממוצע הצבעים של כל הקרניים. זה מוביל למעברים חלקים יותר ולהפחתת קצוות משוננים.

עיצוב ארכיטקטוני

שדות חדשים:

```java
/**
 * Flag for enabling adaptive super-sampling
 *  If true, the camera will use adaptive super-sampling techniques.
 */
private boolean ASS_FLAG = false;  4 usages

/**
 * Flag for enabling anti-aliasing (AA).
 * If true, the camera will use anti-aliasing techniques.
 */
private boolean AA_FLAG = false;  4 usages
```

סט לעידבון הפעלה או ביבוי:

```java
public Builder setAAFlag(boolean flag) {
    camera.AA_FLAG = flag;
    return this;
}
```

הפונקציה private void castRayJ(int nX, int nY, int j, int i) :

- **שולחת את כל הקרניים שנבנו ע"י constructRaysJ(...) ל-ray tracer.**
- **מחשבת את ממוצע הצבעים ומציירת את הפיקסל בהתאם.**

```java
private void castRayJ(int nX, int nY, int j, int i) {  3 usages   naomi1877
    //list of rays for the pixel (j, i)
    List<Ray> rays = constructRaysJ(nX, nY, j, i);

    Color color = Color.BLACK;
    for (Ray ray : rays) {
        color = color.add(rayTracer.traceRay(ray));
    }
    color = color.scale( k: 1.0 / rays.size());
    imageWriter.writePixel(j, i, color);
    // If multithreading is enabled, notify the pixel manager that the pixel is done
    if (pixelManager != null)
        pixelManager.pixelDone();
}
```

הפונקציה public List<Ray> constructRaysJ(int nX, int nY, int j, int i) :

- **מחשבת רשימה של קרניים לפיקסל ספציפי**
- **משתמשת בRandom וב AA_GRID_SIZE כדי ליצור פיזור בתוך הפיקסל.**
- **AA_GRID_SIZE שולט על מספר הקרניים, אם ערכו לדוג 9 הפיזור יהיה(9×9 = 81קרניים)**
- **שליחת קרניים נשלטת על ידי AA_FLAG בשהוא מופעל משתמשים ב castRayJ ובשלא ב castRay**

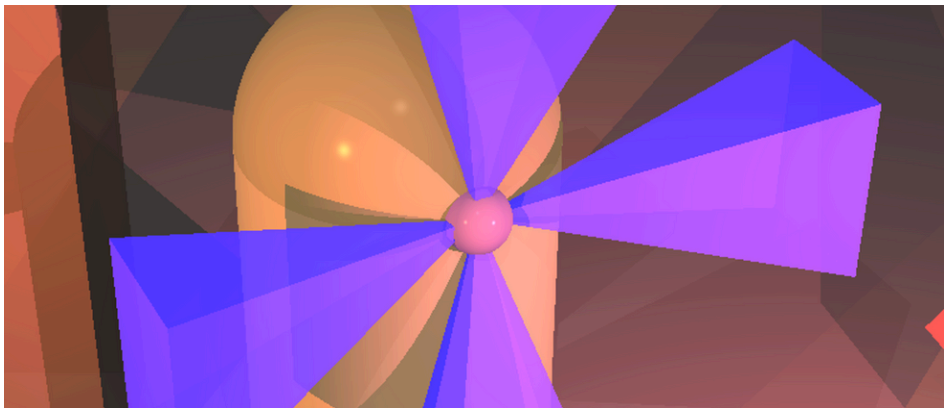
```java
public List<Ray> constructRaysJ(int nX, int nY, int j, int i) {  1 usage  naomi1877
    //list of rays for the pixel (j, i)
    List<Ray> rays = new ArrayList<>();
    Random random = new Random();

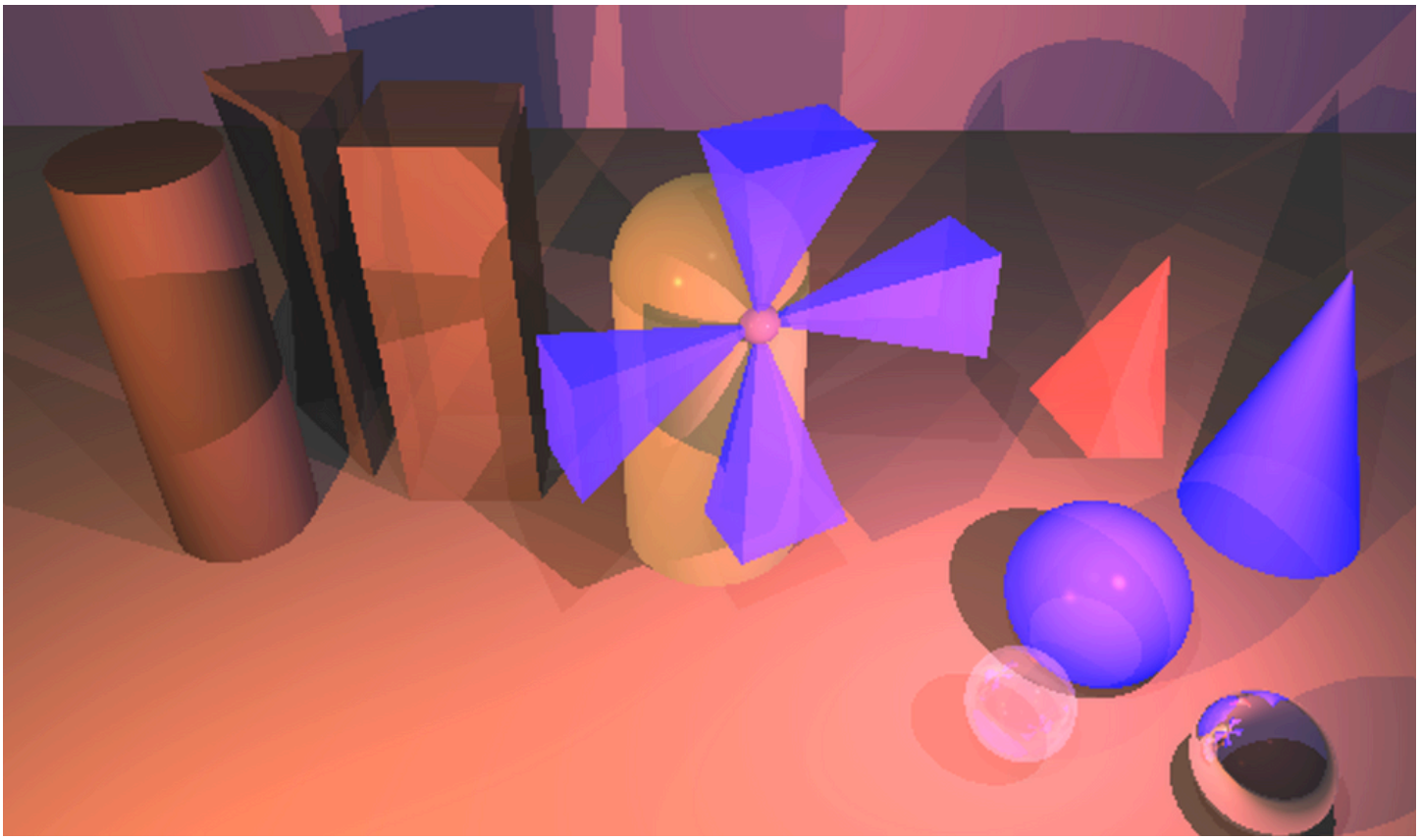
    double Ry = height / nY;
    double Rx = width / nX;
    //calculate the new width and height of the mini pixel
    double stepY = Ry / AA_GRID_SIZE;
    double stepX = Rx / AA_GRID_SIZE;

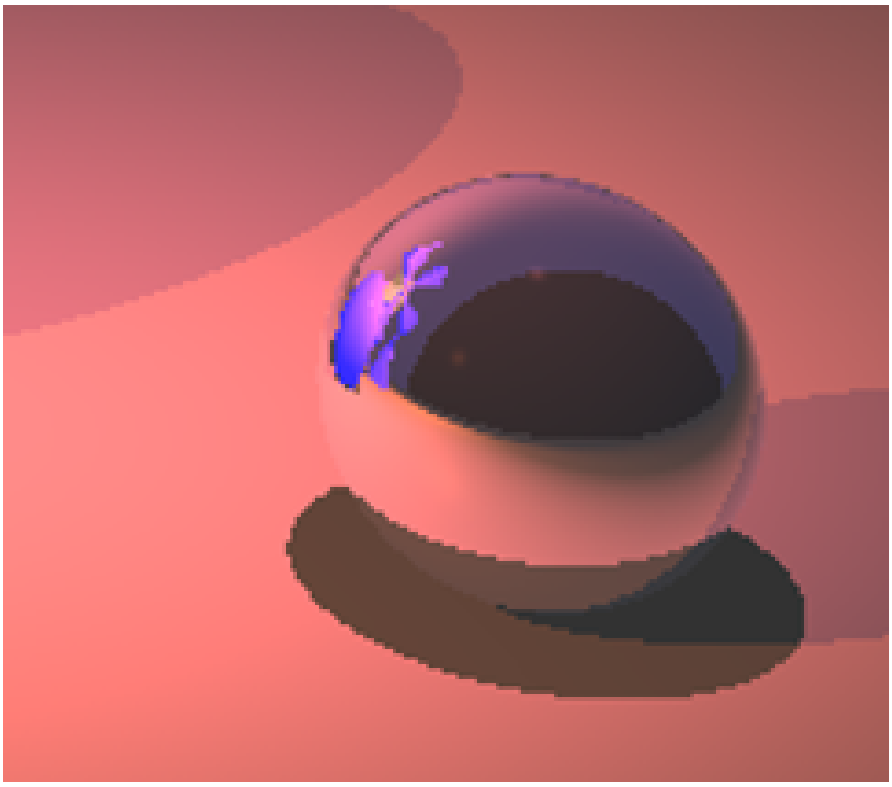
    // Calculate the rays for the pixel (j, i)
    for (int subI = 0; subI < AA_GRID_SIZE; subI++) {
        for (int subJ = 0; subJ < AA_GRID_SIZE; subJ++) {//in mini pixel
            //random offset between 0 and 1
            double jitterY = random.nextDouble(); // Random offset in Y direction
            double jitterX = random.nextDouble(); // Random offset in X direction
            // Calculate the offsets for the sub-pixel location
            double offsetI = (subI + jitterY) * stepY;
            double offsetJ = (subJ + jitterX) * stepX;
            // Calculate the coordinates of the pixel center
            double Yi = -(i - (nY - 1) / 2d) * Ry + offsetI;
            double Xj = (j - (nX - 1) / 2d) * Rx + offsetJ;
            Point pIJ = pCenter;
            if (!isZero(Xj)) pIJ = pIJ.add(vRight.scale(Xj));
            if (!isZero(Yi)) pIJ = pIJ.add(vUp.scale(Yi));
            // Create a ray from the camera position to the sub-pixel location
            Ray ray = new Ray(p0, pIJ.subtract(p0));
            // Add the ray to the list of rays
            rays.add(ray);
        }
    }
    return rays;
}
```

לפני שיפור AA :

setAAFlag(false).

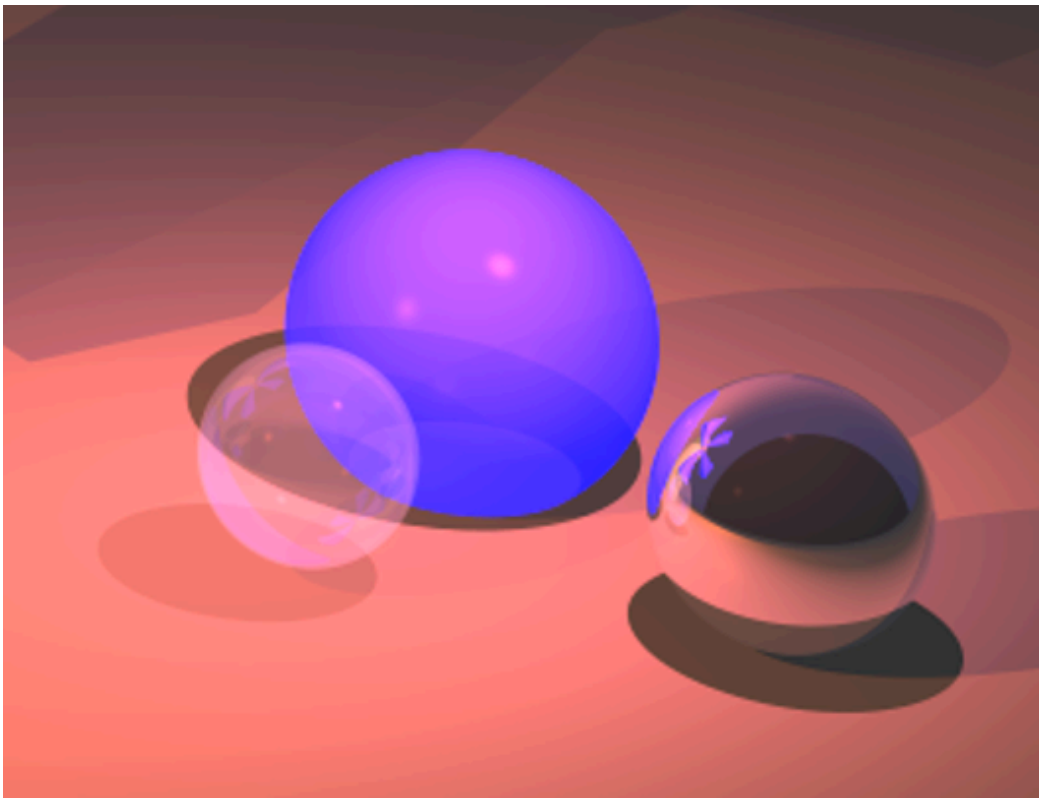


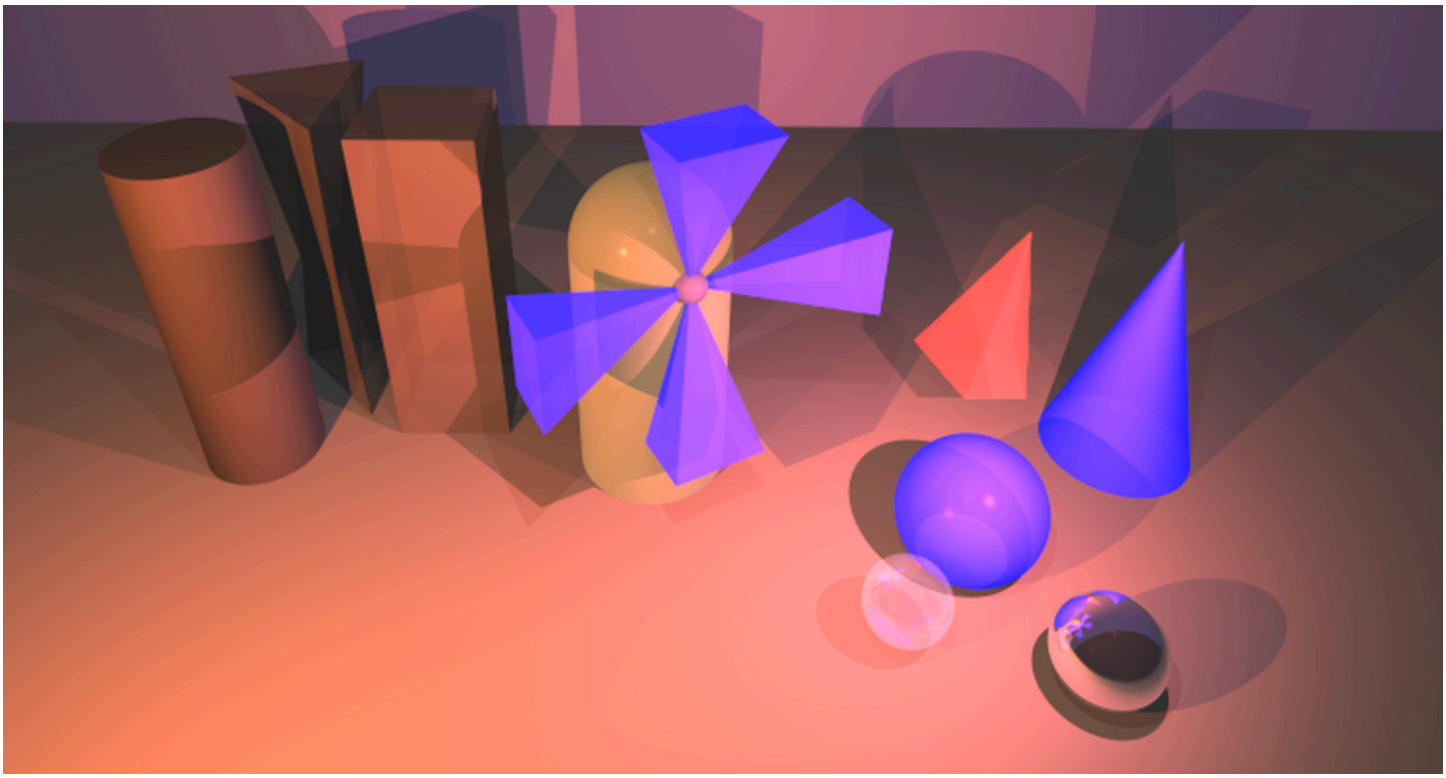




לאחר שיפור AA :

setAAFlag(true).





השפעה ויזואלית

קצוות הגופים היו עם מדרגות בקצוות שפוגעות בנראות, לאחר השיפור המדרגות הוחלקו והתמונה נראת חדה יותר

למה זה עובד טוב

רינדור פיזיקלי של זכוכית = אפקטים אופטיים אמיתיים

דגימה אדפטיבית חוסכת משאבים באזורים אחידים

המערכת המודולרית מאפשרת שליטה קלה על תכונות החומרים

מיני פרויקט 2- Adaptive Super Sampling

הגדרת הבעיה:

בשיפור של AA שלחנו קרניים רבות באופן אחיד לכל פיקסל – גם באזורים שבהם אין בכך צורך. כתוצאה מכך, מתבזבזים משאבים חישוביים על אזורים חלקים שאינם דורשים רמת פירוט גבוהה

השיפור שלנו:

- **שולחים קרן למכז הפיקסל ומחלקים אותו ל4 חלקים**
- **שולחים קרניים לכל רבע, עבור כל תת־פיקסל :**

מחשבים שונות בצבעים

- **אם השונות נמוכה, עוצרים ומחשבים ממוצע**
- **אם השונות גבוהה, ממשיכים לחלק את הפיקסל**

עיצוב ארכיטקטוני:

הפונקציהcastRayASS() יוצרת קרן למרכז הפיקסל ומפעילה את הפונקציה הרקורסיבית

adaptiveSuperSampling() שדוגמת את מרכז הפיקסל ובודקת כמה הצבעים בשאר הפיקסל שונים ממנה ורק כאשר השונות גבוהה תופעל הפונקציה באופן רקורסיבית ותפצל את רבע הפיקסל שוב

```
17  public class Camera implements Cloneable {  64 usages  naomi1877 +1
233
234
235     private void castRayASS(int nX, int nY, int j, int i) {  3 usages  naomi1877
236         double pixelWidth = width / nX;
237         double pixelHeight = height / nY;
238
239         // Calculate the center of the pixel
240         double xJ = (j - (nX - 1) / 2.0) * pixelWidth;
241         double yI = -1 * (i - (nY - 1) / 2.0) * pixelHeight;
242
243         Point pIJ = pCenter;
244         if (!isZero(xJ)) pIJ = pIJ.add(vRight.scale(xJ));
245         if (!isZero(yI)) pIJ = pIJ.add(vUp.scale(yI));
246
247         Color color = adaptiveSuperSampling(pIJ, pixelWidth, pixelHeight, LEVEL); // level is the recursion level for adaptive super-samplin
248         imageWriter.writePixel(j, i, color);
249
250         if (pixelManager != null)
251             pixelManager.pixelDone();
252     }
```

```java
public class Camera implements Cloneable {  64 usages  naomi1877 +1
    /**
     * Performs adaptive super-sampling for anti-aliasing.
     * It recursively samples the pixel and averages the colors based on a threshold.
     * @param center      the center point of the pixel
     * @param pixelWidth  the width of the pixel
     * @param pixelHeight the height of the pixel
     * @param level       the recursion level (depth)
     * @return the averaged color for the pixel
     */
    private Color adaptiveSuperSampling(Point center, double pixelWidth, double pixelHeight, int level) {  2 usages  naomi1877
        if (level == 0) {
            return rayTracer.traceRay(new Ray(p0, center.subtract(p0)));
        }

        double halfWidth = pixelWidth / 2.0;
        double halfHeight = pixelHeight / 2.0;
        Random random = new Random();

        // Calculate the color of center point of the pixel
        Color totalColor = rayTracer.traceRay(new Ray(p0, center.subtract(p0)));
        int count = 1; //


        //the four corners of the pixel
        double[] xSigns = {-1, 1, -1, 1};
        double[] ySigns = {1, 1, -1, -1};
        // Create sample points in the four corners of the pixel
        for (int k = 0; k < 4; k++) {
            double xSign = xSigns[k];
            double ySign = ySigns[k];

            double randX = random.nextDouble() * halfWidth / 2;
            double randY = random.nextDouble() * halfHeight / 2;

            Point samplePoint = center
                    .add(vRight.scale(xSign * (halfWidth / 2 + randX)))
                    .add(vUp.scale(ySign * (halfHeight / 2 + randY)));

            Color sampleColor = rayTracer.traceRay(new Ray(p0, samplePoint.subtract(p0)));


            if (isColorCloseEnough(totalColor, sampleColor)) {
                totalColor = totalColor.add(sampleColor);
                count++;
            } else {
                // recursively sample the quarter
                Point quarterCenter = center//Calculate the center of the quarter
                        .add(vRight.scale(xSign * halfWidth / 2))
                        .add(vUp.scale(ySign * halfHeight / 2));

                Color recursiveColor = adaptiveSuperSampling(quarterCenter, halfWidth, halfHeight, level - 1);
                totalColor = totalColor.add(recursiveColor);
                count++;
            }
        }

        return totalColor.scale(1.0 / count);
    }
```

```java
public class Camera implements Cloneable {  64 usages  naomi1877 +1
    /**
     * Checks if two colors are close enough to be considered equal.
     * This is used for adaptive super-sampling to avoid unnecessary recursion.
     * @param c1 the first color
     * @param c2 the second color
     * @return true if the colors are close enough, false otherwise
     */
    private boolean isColorCloseEnough(Color c1, Color c2) {  1 usage  naomi1877
        final double THRESHOLD = 0.1;

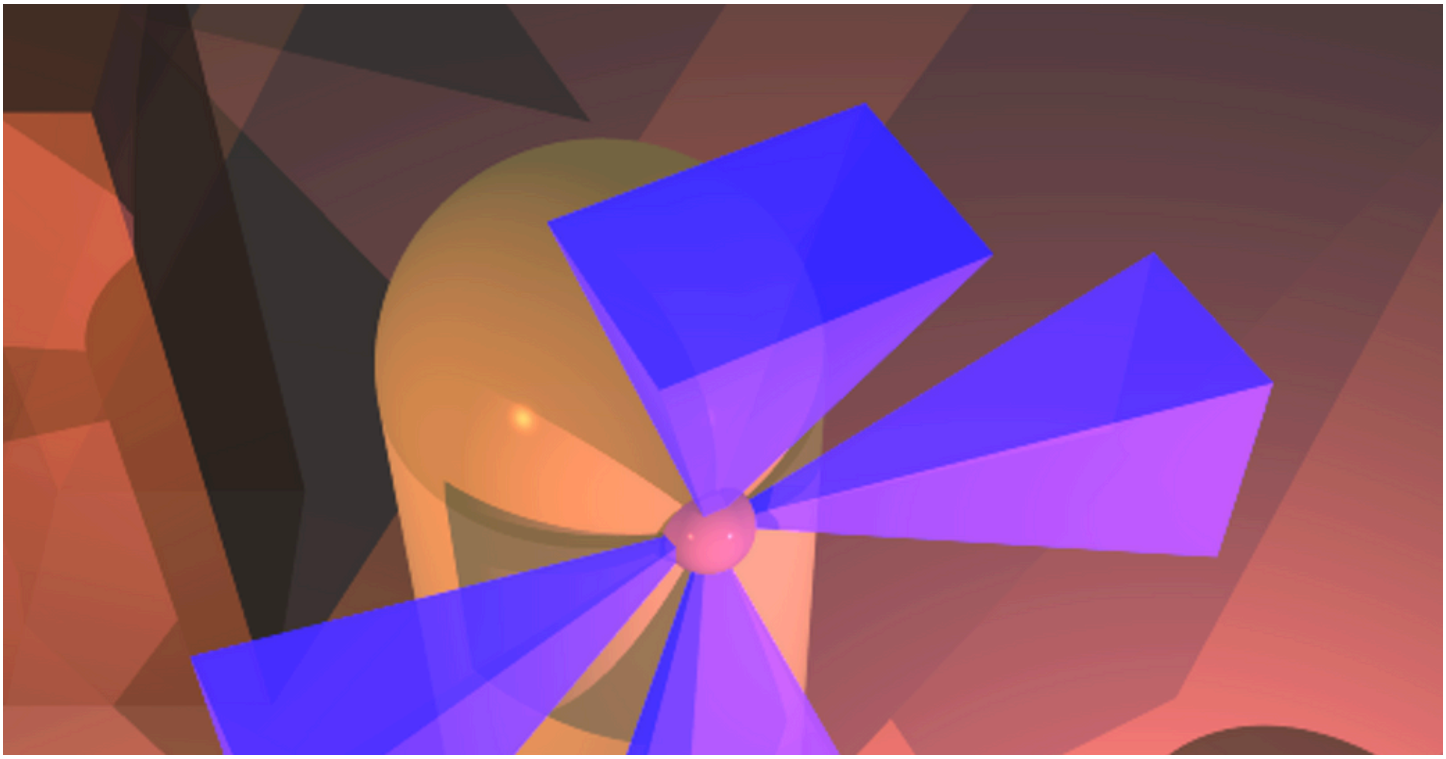
        Double3 diff = c1.getRgb().subtract(c2.getRgb());
        return Math.abs(diff.d1()) < THRESHOLD &&
                Math.abs(diff.d2()) < THRESHOLD &&
                Math.abs(diff.d3()) < THRESHOLD;
    }
```

תיאור גיאומטרי

השיטה פועלת על ריבועי פיקסלים, ומחלקת שוב רק באזורים שבהם יש שונות גבוהה בצבע. כך מושגת יעילות גבוהה: אזורים פשוטים כמעט ואינם מחולקים, ואילו אזורים מורכבים מקבלים יותר קרניים לדיוק מרבי.

הבדל ויזואלי

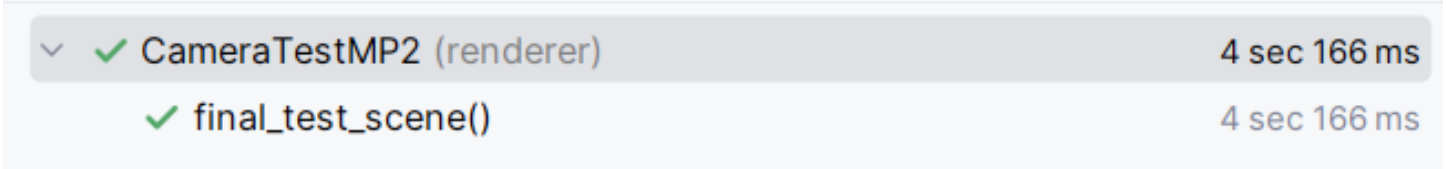
לפני שיפור ASS:
לאחר שיפור ASS:



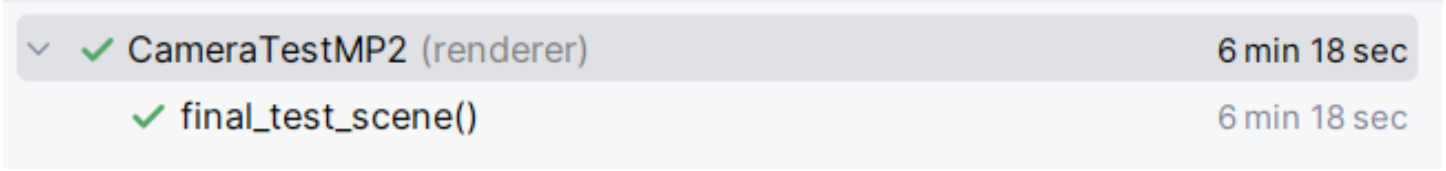
- **פינות חדות יותר**
- **אזורים אחידים מקבלים פחות קרניים - שיפור ביצועים**

זמני ביצועים

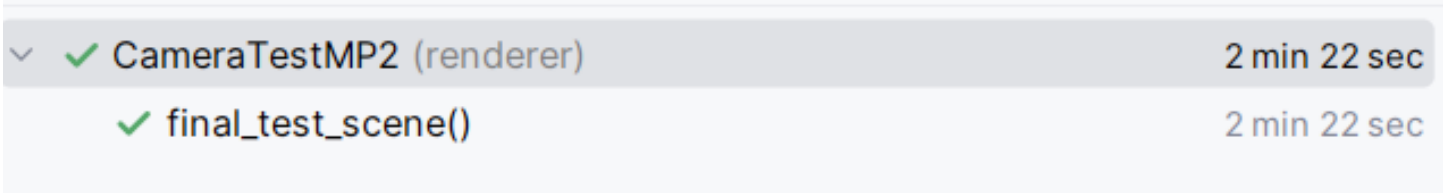
ללא שיפור AA:



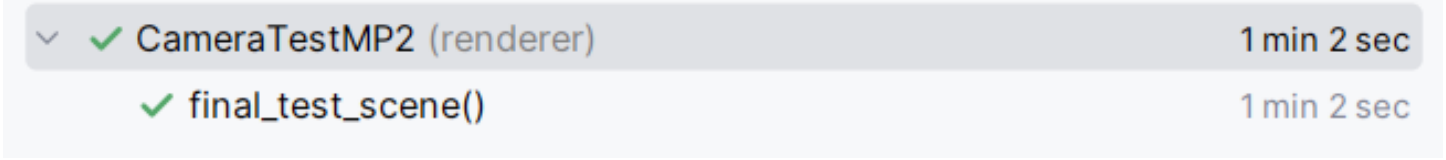
יחד עם AA:



AA+multithreding(-2):



ASS+multithreding(-2):



סיכום מבנה קוד

למה זה עובד טוב

יעיל: מתמקד באזורים שדורשים יותר דיוק

דינמי: מתאים את עצמו למורכבות

איכות גבוהה: קצוות חלקים ושמירה על פרטים

רשימת בונוסים:

- **נורמל לגליל סופי**
- **חיתוך עם גליל אינסופי**
- **חיתוך עם מצולע**
- **חיתוך עם גליל סופי**
- **שימוש בקבצי XML**
- **תמונה עם 10+ עצמים לשלב 7**
- **Jitter עבור AA וכן AAS**